CLASS 3 · RAG · TALK TO YOUR OWN DOCUMENTS

Stop letting your AI *guess*. Give it a library.

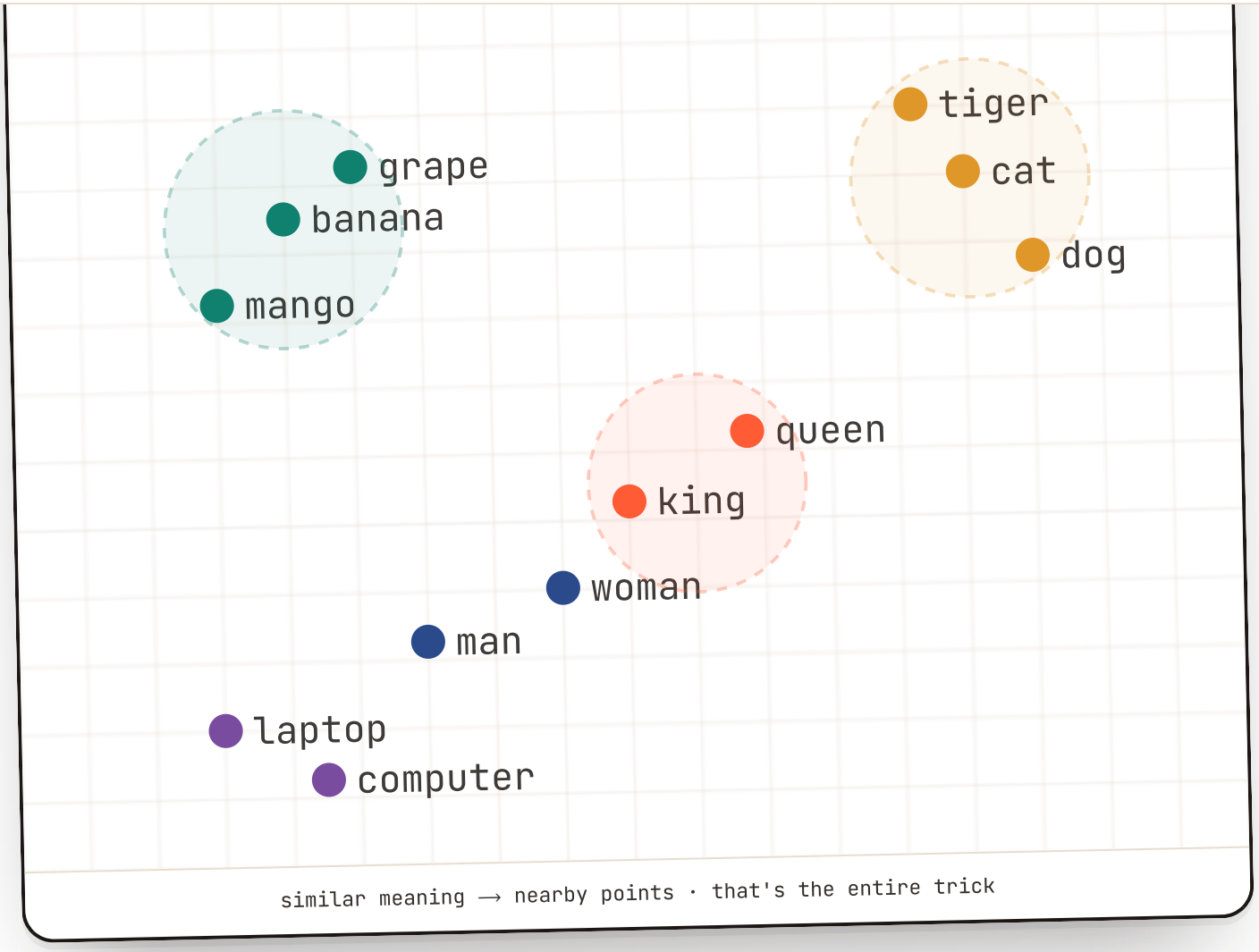
Class 2: your agent could *act* — but it still didn't know **your** data. Today you fix that. You'll build a system that reads your PDFs, finds the right passage for any question, and answers from **real text** — not vibes. It's called **RAG**, and it's how every "chat with your docs" app on the planet actually works. 👉

📚 Embeddings, intuition first ✂️ Chunking, demystified 🗂️ Vector databases

🎯 Reranking + hybrid search 📄 Chat-with-your-PDF, shipped

Scaler Academy — every diagram below is interactive. Drag, click, type along.

vector space, live DEMO



30-SECOND RECAP · CLASSES 1 + 2

What you can already do

- ✓ Call any LLM (OpenAI, Groq, Ollama)
- ✓ Snap a LangChain chain together
- ✓ Pass chat history with placeholders
- ✓ Build an agent that picks tools

🤔 **But your agent has two real problems**

1) It doesn't know *your* stuff — your company's docs, your PDFs, last week's release notes. **2) When it doesn't know, it makes things up** (hallucinates)

🎙 **Speaker note · landing the bridge**

If the shop-assistant agent felt fuzzy last class, this class is the cleanup. Say it plainly: "Last week the model *chose* a tool. Today the tool is its own library — and the magic moves from *which tool* to *how do we find the right page?*." That's RAG, in one sentence.

⏱

PLAN FOR TODAY

Our Class 3 flight plan

Two clean halves: **understand the four pieces of RAG** (embeddings → chunking → vector DB → retrieval) and then **build a real "chat with your PDF" app**. Every concept gets a simulator.

0:00	**Recap + Why RAG?** The 4 problems RAG solves — knowledge cutoff, private data, hallucinations, citations
0:10	**The RAG mental model in 1 picture** Librarian analogy + the 5-box pipeline
0:20	**Embeddings — words become coordinates** The "aha" moment: *king − man + woman ≈ queen*. Live similarity meter.
0:45	**Chunking — cut the book into snippets** Strategies + chunk-size simulator
1:00	**Vector databases — search engine for meaning** Chroma · Pinecone · Qdrant — which one when

	Load → chunk → embed → store → retrieve → answer
1:30	**Make it better — reranking + hybrid search** Bi-encoder ✕ cross-encoder · BM25 ✕ vector
1:45	**Project: Chat with Your PDF** ★ ship it Gradio chat UI + your own doc + public link
2:05	**Ship it to LinkedIn** ★ ship it Caption template + post — today 🎉
2:15	**Peek ahead — Agentic RAG** Where this leads next class

BLOCK 14 · ~10 MIN · THE HOOK

14

Why does RAG *exist?*

An LLM is a brilliant intern who finished training a year ago and was never allowed inside your office. RAG fixes both — fresh data, *and* your private data — by handing the intern a relevant page **at question-time**.

📅 Knowledge cutoff

The model's training ended months ago. It doesn't know yesterday's RBI rate cut or last week's Budget numbers.

✓ RAG: fetch today's article

🔒 Private / internal data

It has never seen your HR policy, your contracts, your product wiki, or your customer tickets — and you can't paste them all into one prompt.

✓ RAG: search your docs

When unsure, the model invents confident-sounding answers. Bad in customer support, dangerous in legal, fatal in healthcare.

✓ RAG: ground in real text

Ask "where did you get that?" and a raw LLM has nothing. Real products need *"see source: page 14"*.

✓ RAG: return the source chunk



The 1-line definition

RAG = Retrieval-Augmented Generation. Before the model answers, *you* retrieve the most relevant snippets from your own data and stuff them into the prompt. The model then answers *from* those snippets. No retraining. No fine-tuning. Just open-book exam instead of closed-book.



INDUSTRY SPOTLIGHT · WHERE YOU'VE SEEN THIS ALREADY

Almost every "AI feature" you've used is RAG underneath

Notion AI answering from your workspace, Glean searching your company's apps, Perplexity citing sources, Intercom's Fin support bot, ChatGPT's "search the web", Cursor finding code in your repo, even the new "ask about this PDF" button in your browser — same pattern, every time. Master this, and you can clone any of them.

Notion AI Glean Perplexity Intercom Fin Cursor ChatGPT Search



Quick reality-check (mention in class)

People sometimes ask: "Why not just fine-tune the model on our data?" Three reasons it's usually wrong: **(1)** expensive and slow to redo every time data changes, **(2)** still can't cite sources, **(3)** still hallucinates. RAG is faster, cheaper, traceable, and almost always the right first move.

BLOCK 16 · ~25 MIN · THE "AHA" OF THE WHOLE CLASS

16

Embeddings: words become *coordinates*

Before we can "search by meaning", we need a way to **turn meaning into numbers**. That's an *embedding*. And once you see what it does, every confusing thing about RAG suddenly makes sense.

DEFINITION · KEEP THIS IN YOUR HEAD

An **embedding** is a list of numbers (a *vector*) that captures the meaning of a piece of text. Similar meanings ⟶ similar numbers ⟶ nearby points in space. Different meanings ⟶ far apart.

Imagine a 2D world first — height & weight

even 3072 dimensions — way more than we can draw — but the principle is identical: *similar ⟶ close, different ⟶ far.*

📊 **"Why so many dimensions?"**

Because meaning is rich. Two words can be similar in many ways at once — *topic, tone, formality, language, sentiment*. Each dimension captures a different axis of similarity. 768 isn't arbitrary; it's just enough to separate millions of distinct ideas.

The *aha*: embedding math actually works

This is the demo that turned a generation of engineers into believers. Trained on enough text, vectors carry real relationships you can do arithmetic on. Pick a side and watch:

`king - man + woman ≈`

king − man + woman | paris − france + india | tokyo − japan + germany | walking − walk + run

Click a different equation. Each is a *real result* from GloVe / Word2Vec embeddings — no tricks. The model never saw any rule like "queen is the female king" — that pattern just *emerges* from the geometry of meaning.

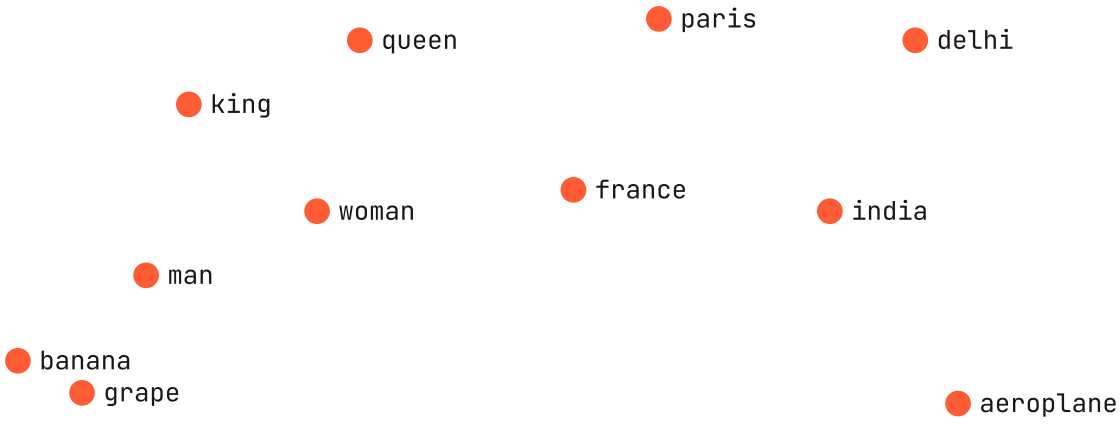
🤯 **Why this is the foundation of everything today**

If *woman* and *queen* can be found by simple arithmetic, then "find the chunk most similar to my question" is also just arithmetic — fast, scalable, and runs on commodity hardware. Every RAG system uses exactly this idea. **Search by meaning = nearest point in vector space.**

A real-world plot of word vectors squished from 768 dimensions down to 2 for visualization. Click pairs and notice the *direction* connecting them is the same:

LIVE SIM Vector space mini-map

click the pairs · watch the arrows align



👑 king ↔ queen / man ↔ woman

🏛 paris ↔ france / delhi ↔ india

🍎 banana ↔ grape (close) vs aeroplane (far)

↺ clear

"capital-of" is encoded as the **direction** from country → capital. Same arrow length, same angle, anywhere on the map. That direction *is* the relationship.

The score we use: *cosine similarity*

Two vectors close together = the *angle* between them is small. The cosine of that angle is our score. It ranges from **-1 (opposite) to +1 (identical)**. Practically: **above 0.7 means "very similar"**, below 0.3 means "barely related". No math needed to use it — just remember: *bigger number = more similar*.

Two clock hands pointing at the same time ⟶ angle = 0 ⟶ cosine = 1 ⟶ perfectly similar. Pointing at 12 and 6 ⟶ angle = 180° ⟶ cosine = −1 ⟶ opposites. We don't care how *long* the hands are, only the angle. That's why text length doesn't break the score.

Try it — the similarity meter

Type any two sentences. The meter shows the cosine similarity our system would compute (this in-browser simulation uses a small built-in semantic table, so results feel realistic for common topics).

LIVE SIM **Cosine similarity meter** try opposites, synonyms, totally unrelated pairs

Sentence A

A cat is sleeping on the couch.

Sentence B

A kitten is napping on the sofa.

synonyms | paraphrase | same city, different names | related concepts | unrelated | totally unrelated

0.88

COSINE SIMILARITY

Near-synonyms — same meaning, different words.

How do we actually get an embedding? One function call.

You don't have to train anything — somebody else already did. Open-source models from Hugging Face (`sentence-transformers`) or APIs from OpenAI / Cohere give you

embeddings.py

```python
# pip install sentence-transformers
from sentence_transformers import SentenceTransformer

model = SentenceTransformer("all-MiniLM-L6-v2")   # ① free, fast, 384 dims

vec = model.encode("A cat is sleeping on the couch")  # ② that's an embedding!
print(vec.shape)          # → (384,)  — 384 numbers
print(vec[:5])           # → [-0.05, 0.12, 0.41, -0.08, 0.22] (something like that)

# to compare two pieces of text → encode both → take cosine similarity
from numpy import dot
from numpy.linalg import norm

v1 = model.encode("A cat is sleeping on the couch")
v2 = model.encode("A kitten is napping on the sofa")
similarity = dot(v1, v2) / (norm(v1) * norm(v2))   # cosine
print(similarity)        # → 0.87 (very similar 🥕)
```

🔍 DECODER

① **all-MiniLM-L6-v2** is a tiny but excellent open-source embedding model — runs on your laptop, no API key needed. For production, look at `BAAI/bge-large-en-v1.5` (best English) or OpenAI's `text-embedding-3-small` (paid, very good multilingual).

② `.encode("text")` returns the vector. That's the whole API surface — one call, 384 numbers back. Now your text is searchable by meaning.

🎙️ **Speaker note · how to pitch this section**

This is the "physics" of RAG. Don't rush. Spend a full 5 minutes letting them play with the meter and the king-queen equation. Once they internalize *"similar meaning = nearby vector"*, everything else in this class is bookkeeping.

Vectors = coordinates

Every chunk of text gets a point in space.

Cosine similarity

Score from −1 to +1. Bigger = closer.

Math you can do

king − man + woman ≈ queen. Really.

⚡

One function call

`.encode(text)` — that's it.

BLOCK 17 · ~15 MIN · THE UNGLAMOROUS PART THAT DECIDES EVERYTHING

17

Chunking: cut the book into *snippets*

Before we embed anything, we have to cut it up. You can't embed a 200-page PDF as one vector — meaning gets averaged away to mush. So we split it into *chunks*. **How** you chunk decides how good your RAG actually is. Engineers underestimate this; the best ones obsess over it.

🍕

The pizza-slice analogy

One whole pizza is hard to share — too big. Cut it into 100 confetti-sized bits and nobody can taste anything. **Slice sizes matter.** Chunks are pizza slices: too big and you lose precision (the relevant bit is buried), too small and you lose context (each crumb is meaningless).

The 4 chunking strategies worth knowing

Every N characters or tokens — done. Brain-dead simple, fast, your default. *Downside:* chops mid-sentence, mid-table, mid-thought.

Best for: prototypes, blog posts

Try to split on paragraphs first; if too big, split on sentences; if still too big, on words. Smart fallback ladder. The default in LangChain.

Best for: most real apps

Semantic

Embed every sentence; group consecutive sentences that "talk about the same thing". Chunks follow meaning, not size.

Best for: long flowing prose

Structure-aware

Use the document's own structure — markdown headings, code blocks, HTML sections. Each section becomes a chunk.

Best for: docs, code, wikis

Watch all 4 strategies — same text, four different cuts

Same source passage, four ways to chop it. Switch tabs and watch **where the cuts land** and **how chunks change shape**. The differences *are* the lesson — pure text won't make this click; the picture will.

LIVE SIM 4 chunking strategies · same source click tabs · compare cuts side by side

SOURCE · A TINY MARKDOWN DOC WITH 3 TOPICS

Bengaluru Overview

Tech Industry
Bengaluru is known as India's Silicon Valley. Tech parks like Electronic City and Whitefield host thousands of tech companies. Major firms include Infosys, Wipro, and TCS.

Climate
The city sits at 920 meters altitude. This gives it pleasantly cool weather year-round. Average temperatures rarely exceed 30 degrees.

here. Filter coffee shops dot every street corner.

Fixed blind char-count

Recursive respect boundaries

Semantic group by topic

Structure follow headers

Chunk size: 120 characters · try 80 to see the difference

```
# Bengaluru Overview

## Tech Industry
Bengaluru is known as India's Silicon Valley. Tech parks like Electronic City and
```
120 chars

```
 Whitefield host thousands of tech companies. Major firms include Infosys,
Wipro, and TCS.

## Climate
The city sits at
```
120 chars

```
920 meters altitude. This gives it pleasantly cool weather year-round. Average
temperatures rarely exceed 30 degrees.

#
```
120 chars

🔨 # Food
Bengaluru's food scene is legendary. South Indian classics like masala dosa and idli thrive here. Filter coffee s

120 chars
mid-word ✂

🔨 hops dot every street corner.

29 chars
mid-word ✂

5 chunks **102** avg chars **2** mid-word cuts 🔨

Chunks marked 🔨 were split *mid-word*; context gets fragmented. Cheap to write (one for-loop), but bad for retrieval. Prototypes only.

Try this sequence in class: start on *Fixed* at size 80 → count the 🔨 mid-word cuts → flip to *Recursive* at the same size → they're all gone → flip to *Semantic* → notice three chunks emerge that align perfectly with the three topics → flip to *Structure* → see how the markdown `##` headers do the work for free.

THE RECALL VS PRECISION TRADEOFF · KEEP THIS IN MIND

Big chunks → high recall (the answer is probably *in* there), but low precision (lots of fluff around it). **Small chunks** → high precision (the chunk is exactly the answer), but low recall (the relevant bit might be split across two chunks and you only pulled one). Most teams sweep chunk size as their first RAG tuning knob.

One line of real code

chunk.py

```python
from langchain_text_splitters import RecursiveCharacterTextSplitter

splitter = RecursiveCharacterTextSplitter(
    chunk_size=800,        # aim for ~800 chars per chunk
    chunk_overlap=100,     # adjacent chunks share 100 chars (context glue)
)

chunks = splitter.split_text(your_long_document)
print(len(chunks))    # → e.g. 47 chunks ready to embed
```

Why `chunk_overlap` ?

If your question's answer sits exactly at a chunk boundary, you'd miss it. Overlap (50–100 chars) makes sure every sentence appears in at least one chunk fully. Cheap insurance.

BLOCK 18 · ~10 MIN

18

Vector databases: a search engine for *meaning*

You have thousands of embeddings. For every question, you need the top-k nearest ones — *fast*. That's what a vector database does, and that's **all** it does.

📁 **One-line definition**

A vector database is a *search engine* where instead of "find documents containing this word", the query is "find vectors closest to *this* vector". Same idea as Google, swapped engine.

The three names you'll hear all the time

★ TODAY

Chroma

Open-source, runs in-process (no server!), one `pip install`. Perfect for prototypes & up to ~10M vectors. **What we'll use today.**

Pinecone

Fully managed SaaS. Zero ops, scales to billions, but paid. The boring-and-reliable choice for production.

Qdrant

Open-source *and* production-grade. Self-host or use their cloud. Great middle ground when you outgrow Chroma.

🎯 **Picking one (don't overthink)**

Prototyping or under 1M chunks? **Chroma**. Want zero ops & have a budget? **Pinecone**. Need to self-host at scale? **Qdrant** or **Weaviate**. The good news: LangChain wraps all of them with the same interface, so swapping later is a one-line change.

A VECTOR DB IS JUST THESE THREE CALLS

1. `db.add(documents, embeddings)` — store chunks and their vectors. (Indexing.)
2. `db.query(query_vector, k=3)` — return the 3 chunks whose vectors are closest. (Retrieval.)
3. `db.delete(ids)` — remove chunks when a doc is deleted. That's it. Three calls.

🏭 INDUSTRY SPOTLIGHT · HOW IT STAYS FAST

HNSW — the trick behind every vector DB

Searching billions of vectors naively means computing billions of distances per query. **HNSW** (Hierarchical Navigable Small World — a 2018 algorithm) builds a "ladder" graph so each query takes only ~log(N) hops. Result: sub-10ms lookups on 100M vectors. You'll never write this yourself — every vector DB ships it built-in.

`Chroma · HNSW` `Pinecone · proprietary` `Qdrant · HNSW` `FAISS · IVF + HNSW`

BLOCK 20 · ~15 MIN · 80% OF RAG QUALITY LIVES HERE

20

Make it better: *rerank* + *hybrid* search

Both are easy to add.

Problem 1: the top-5 from vector search isn't the *best* 5

Embedding similarity is fast, but it sometimes ranks shallow word-matches above deep semantic matches. So we use a **two-stage retrieval** — a fast first pass to narrow down, then a slow accurate pass to pick the real winners.

Imagine hiring for one open role — with 1,000 applicants

You can't interview 1,000 people (it would take a year). You also can't pick someone by gut feel from a stack of resumes (you'd hire badly). So you do **two passes**: a fast *resume scan* to shortlist 25, then a real *30-minute interview* with each of those 25. **Bi-encoder and cross-encoder are exactly these two passes.**

Bi-encoder

LIKE A RESUME SCAN

Reads the query and the document **separately**, turns each into a vector, then compares the two vectors with cosine similarity. The encoder *never sees them together* — it judges each "card" alone.

- **Fast** — sub-10ms over millions of docs
- **Pre-computable** — encode all your docs once, store forever, only encode the query at runtime
- **Shallow** — misses subtle relevance because the model never compares them side-by-side

→ use it to grab top 50–100 candidates

Cross-encoder

LIKE A 30-MIN INTERVIEW

Feeds the query **and** the document into one transformer *together*, lets the model attend to both at once, then outputs a single relevance score. The model can compare them token by token, weigh trade-offs, spot nuance.

- **Way more accurate** — sees query↔doc word interactions directly
- **Slow** — full transformer run per (query, doc) pair
- **Not pre-computable** — each score is pair-specific, you can't cache anything

→ use it to rerank those 50 → top 3

ALL YOUR CHUNKS → ~10 ms → CANDIDATES → ~200 ms → TO THE LLM

Funnel total: ~210ms — fast enough for live chat, accurate enough to beat raw vector search by miles.

REMEMBER THIS — THE TWO-STAGE PATTERN IS UNIVERSAL

Stage 1 (bi-encoder): grab top 50-100 candidates from the vector DB. Fast.

Stage 2 (cross-encoder): re-score those candidates with a smarter model. Slow per item, but you're only re-scoring 50, not 50 million. Keep the top 3-5 for the LLM.

See reranking flip the order

Same query, same candidates. *Left*: what cosine similarity returned. *Right*: what a cross-encoder reranker returns. Press the button and watch the real answer rise:

LIVE SIM Reranker · before vs after

query: "how do I return a defective product?"

STAGE 1 · VECTOR SEARCH (TOP 5)

1. Our shipping is fast and reliable for all returns.	0.78
2. Refund policy: items can be returned within 30 days.	0.74
3. Return shipping labels are emailed after request.	0.71
4. For damaged or defective items, replacements are sent free of cost.	0.69

→

STAGE 2 · CROSS-ENCODER RERANK

Press *Run* to see the cross-encoder rerank.

▶ Run reranker ↺ Reset

Notice doc #3 — it never says the word "return" but it's clearly the most relevant. Pure vector search ranks it #4; cross-encoder lifts it to #1. *That* is why we rerank.

The code to add reranking is laughably short:

rerank.py

```python
from sentence_transformers import CrossEncoder

reranker = CrossEncoder("cross-encoder/ms-marco-MiniLM-L6-v2")   # free, fast

def retrieve_with_rerank(question, top_k=3):
    candidates = db.similarity_search(question, k=25)        # grab 25 cheap candid
    pairs = [(question, c.page_content) for c in candidates]
    scores = reranker.predict(pairs)                          # cross-encoder score
    return [c for _, c in sorted(zip(scores, candidates), reverse=True)[:top_k]]
```

Problem 2: sometimes you need the *exact word*

Pure semantic search ignores keywords. Ask "what's the price of **SKU-4429**?" and the embedding doesn't care about that specific code — it just sees "price" and "product". For codes, names, IDs, dates — **keyword search beats semantic** hands down. The fix isn't to pick one or the other. It's to use *both*, with very different strengths.

Two librarians, very different superpowers

Remember our librarian from Block 15? She has **two assistants** who search the stacks completely differently. One is a literalist who lives for exact words. The other is a philosopher who lives for meaning. Send your query to the right one and you

Librarian A · BM25

THE LITERALIST

Obsessed with **exact words**. Type *"SKU-4429"* or *"Article 21"* and she'll find every single doc containing those exact characters. But ask for *"running shoes"* and she'll skip the doc titled *"marathon footwear"* — different words, even though same meaning.

- ✅ **Brilliant at** product codes, SKUs, model numbers, version strings
- ✅ **Brilliant at** proper nouns, dates, legal references, technical jargon
- ❌ **Blind to** synonyms, paraphrase, intent — words are just characters to her

→ `catches what vector misses`

Librarian B · Vector

THE PHILOSOPHER

Obsessed with **meaning**. Ask for *"running shoes"* and she surfaces *"marathon footwear"*, *"jogging trainers"*, even *"sneakers for athletes"*. But ask for *"SKU-4429"* and she shrugs — random letters and numbers are just noise to her.

- ✅ **Brilliant at** natural-language questions, "what's this about?"
- ✅ **Brilliant at** fuzzy queries, paraphrase, multilingual matching
- ❌ **Blind to** exact codes, IDs, technical strings — drowns them in averages

→ `catches what BM25 misses`

Hybrid = both librarians on the case

Send the same query to *both* librarians at the same time. Each ranks the docs by their own logic. You then **merge their two ranked lists into one** using a tiny formula called **Reciprocal Rank Fusion (RRF)**:

⚡ **Reciprocal Rank Fusion in one line**

For each doc, final score = `1 / (60 + rank in BM25) + 1 / (60 + rank in Vector)` . Docs that *both* librarians ranked highly bubble to the top. Docs only one of them liked still get a fair shot. No tuning, no thresholds, no magic numbers (well, 60 — but it almost never matters). *That's it.* Used by almost every production RAG system in the wild.

Real user queries are *messy* — mostly natural language, but sprinkled with technical terms, product codes, names, or jargon the embedding model has never seen. Hybrid covers both halves of every messy query automatically: the natural-language part goes to Vector, the technical-term part goes to BM25, and RRF stitches the answers together. **You're not picking sides — you're using each tool for what it's actually good at.**

See it on a real query — *"I want running shoes"*

Same query, three rankings. Watch how Librarian A and Librarian B return different top picks — and how Hybrid merges them. Press the button to see who wins:

LIVE SIM BM25 vs Vector vs Hybrid same query, three rankings

BM25 KEYWORD

press ▶ Compare

Counts word overlap. Loves "shoes" appearing literally.

Vector SEMANTIC

press ▶ Compare

Understands "running" means jogging — even if the word "shoes" isn't there.

Hybrid BLENDED

press ▶ Compare

▶ Compare all three

🏭 INDUSTRY SPOTLIGHT · THIS IS WHAT THE LEADERBOARD CHASES

Every "+10% retrieval quality" paper is one of these tricks

Reranking + hybrid search are the two single biggest quality wins in RAG. Add them and you go from "demo works" to "production works". Almost every benchmark in the MTEB leaderboard uses some combination. You now know the playbook.

`BM25 + Vector` `Cross-encoder rerank` `Reciprocal rank fusion`

`Cohere Rerank API`

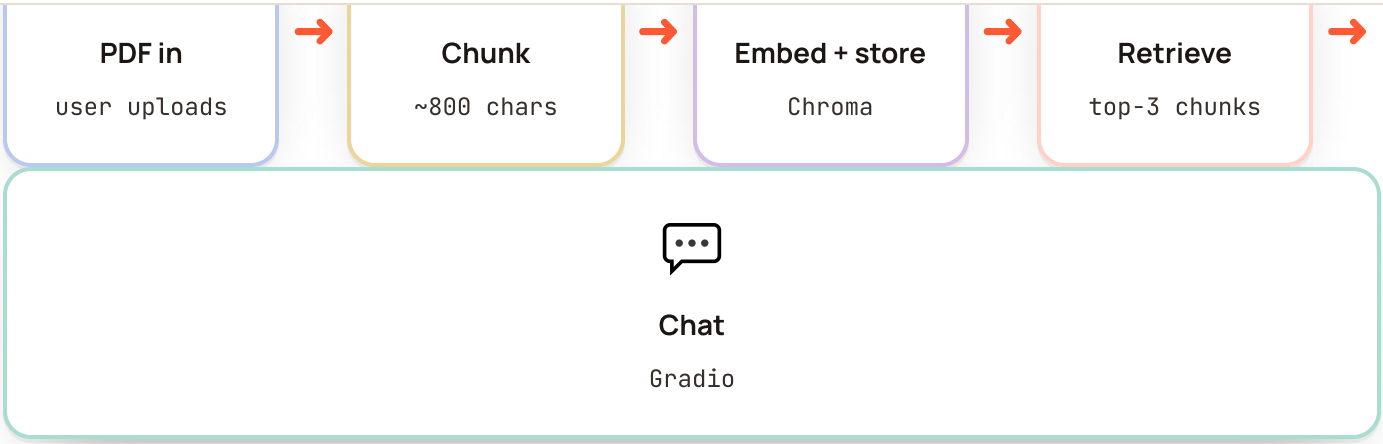
BLOCK 21 · ~20 MIN · 🏁 THE BUILD

21

Mini-Project: Chat with *your* PDF

Time to ship. Upload any PDF — your resume, a research paper, an annual report, your college notes — and chat with it. Every answer cites the page it came from. This is the project that gets **"how did you build this?"** in your LinkedIn DMs.

The full app — 4 files



STEP 1 — LOAD THE PDF & INDEX IT ONCE

pdf_chat.py · part 1

```
# pip install langchain langchain-openai langchain-chroma langchain-huggingface \
#             langchain-community pypdf sentence-transformers gradio python-dotenv

from langchain_community.document_loaders import PyPDFLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_huggingface import HuggingFaceEmbeddings
from langchain_chroma import Chroma

def build_index(pdf_path):
    pages    = PyPDFLoader(pdf_path).load()                    # ① read all pages
    splitter = RecursiveCharacterTextSplitter(chunk_size=800, chunk_overlap=100)
    chunks   = splitter.split_documents(pages)                 # ② chunk them
    embedder = HuggingFaceEmbeddings(model_name="all-MiniLM-L6-v2")
    db       = Chroma.from_documents(chunks, embedder)         # ③ in-memory store
    return db
```

STEP 2 — ANSWER WITH RETRIEVAL + CITATIONS

pdf_chat.py · part 2

```
from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate
from dotenv import load_dotenv
```

```
model = ChatOpenAI(model="gpt-4o-mini", temperature=0)

prompt = ChatPromptTemplate.from_template("""
You are a helpful PDF assistant. Answer the question using ONLY the context below.
If the context doesn't contain the answer, say "I couldn't find that in the documer
After your answer, list the page numbers you used as: Sources: page X, page Y.

Context:
{context}

Question: {question}
""")

def ask(db, question):
    chunks  = db.similarity_search(question, k=4)
    context = "\n\n".join(
        f"[page {c.metadata['page']+1}]] {c.page_content}" for c in chunks)
    chain = prompt | model
    return chain.invoke({"context": context, "question": question}).content
```

STEP 3 — GIVE IT A FACE WITH GRADIO

pdf_chat.py · part 3

```
import gradio as gr

state = {"db": None}                    # ① remember the index across turns

def upload(pdf):
    state["db"] = build_index(pdf.name)   # ② re-index whenever a new PDF arrives
    return "✅ PDF indexed! Ask me anything about it."

def chat(message, history):
    if state["db"] is None:
        return "Please upload a PDF first 📄"
    return ask(state["db"], message)

with gr.Blocks(title="📄 Chat with your PDF") as demo:
    gr.Markdown("## 📄 Chat with your PDF (powered by RAG)")
```

```
    pdf.upload(upload, inputs=pdf, outputs=status)
    gr.ChatInterface(fn=chat)

demo.launch(share=True)                    # share=True → public link!
```

🔍 DECODER · THE 3 SMALL CHOICES THAT MATTER

① `state` is a plain dict that survives across Gradio events. Without it, every chat message would re-index the PDF from scratch (slow!).

② Re-indexing only on upload means typing is instant. This is the same indexing/querying split from Block 15.

③ We pass `page` from the chunk metadata into the prompt — that's how the model knows which page to cite. **Metadata is RAG's superpower.**

RUN IT

```
bash — your project folder

$ python pdf_chat.py

Running on local URL:  http://127.0.0.1:7860
Running on public URL: https://abcd12.gradio.live   # ← share this on LinkedIn!
```

▶ **Upload *any* PDF**

Your resume, the Indian Constitution, your company's HR policy, last quarter's earnings call transcript. Anything text-based.

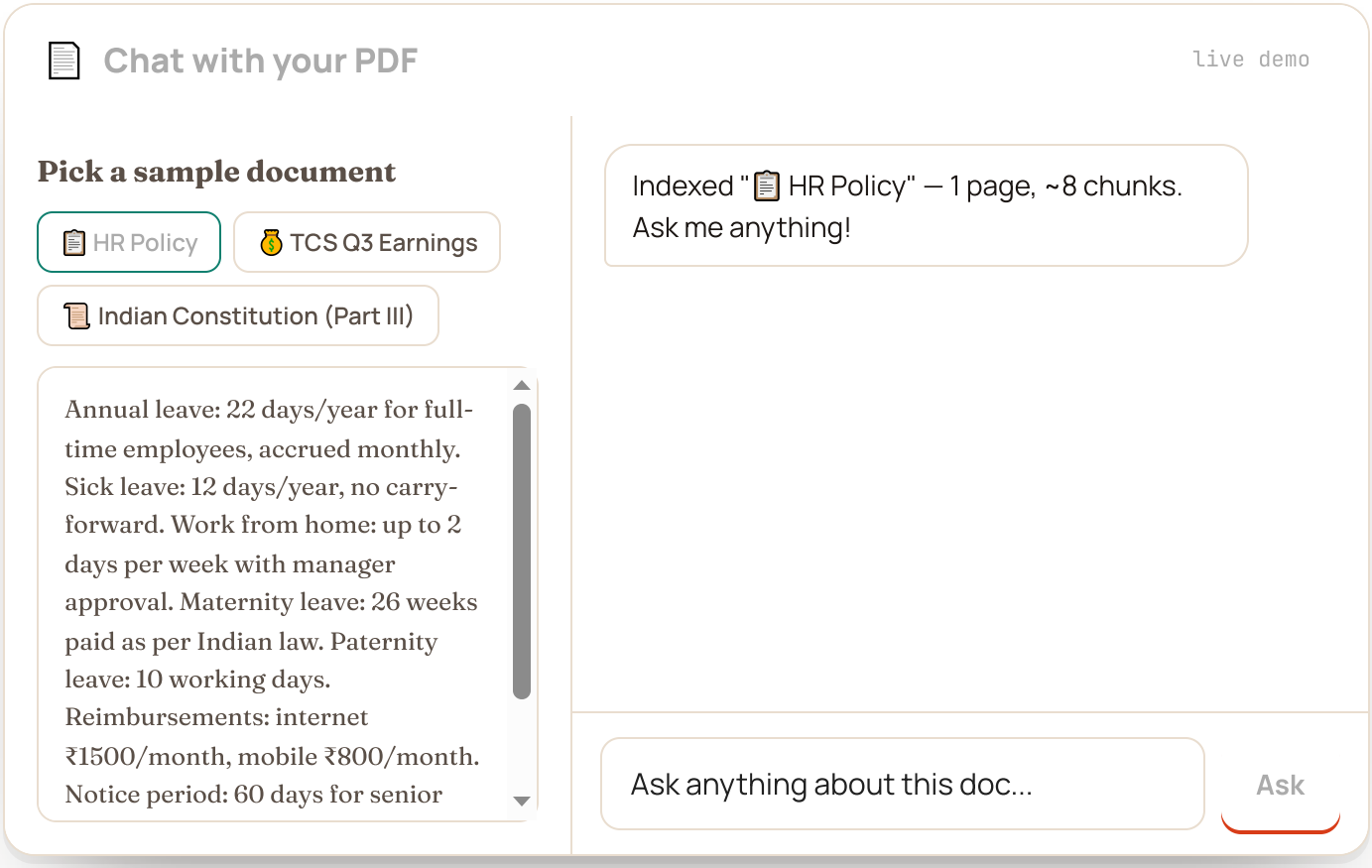
▶ **Ask 3 questions**

One factual ("when was X founded?"), one summary ("what's the main argument?"), one tricky ("compare X and Y"). Notice the citations.

▶ **Try a question that's NOT in the doc**

Try the working version 👇

A live in-browser version with three pre-indexed sample documents — click a PDF, ask a question, see RAG retrieve the right chunk before answering:



Try: How many leaves do I get? Can I work from home? What is the notice period? What is the maternity policy?

Simulated in-browser with a tiny semantic-matching layer so it runs key-free. Your real `pdf_chat.py` uses actual embeddings + GPT — same flow, same feel, larger brain.

INDUSTRY SPOTLIGHT · THIS *IS* THE CANONICAL AI PRODUCT

You just built the most-shipped AI app of 2024–26

"Chat with [your docs / your PDF / your codebase / your Notion]" is the single most common AI feature on the market today — and almost all of them are this exact pattern

Notion AI Glean Claude Projects Cursor @docs Perplexity Spaces

22

BLOCK 22 · ~10 MIN · 📣 SHIP IT

Ship it to LinkedIn — *today*

"I built a RAG system" is an instant credibility upgrade — most people who post about AI have never actually built one. The shipping ritual continues: clip, caption, post, reply.

The 3-step ship checklist (same as Class 1 & 2)

- **Record a 20-second clip**
  Upload your PDF on screen, ask 2 questions, show the *"I couldn't find that"* moment. That's the magic.
- **Use the caption template below**
  Fill in the blanks. Mention what PDF you tested with — specifics make it land.
- **Post + reply for 1 hour**
  Tag *#AIEngineering*, *#RAG*, *#100DaysOfCode*. Replying to every comment in the first hour is the algorithm's favourite signal.

Steal this caption

📄 I just built *"Chat with your PDF"* — and it actually *understands* the document.

Upload any PDF (I tested with [your doc here]) and it answers questions from the real text, with page citations. If the answer isn't in the doc, it says so instead of making something up. 😲

The trick is called **RAG** (Retrieval-Augmented Generation):
🔪 Chop the doc into chunks
📍 Turn each chunk into a vector (a "coordinate of meaning")
🔎 For every question, find the 3 nearest chunks
📋 Hand them to the LLM and let it answer from *real text*

The most surprising thing? You can do **math on meaning**. *king − man + woman ≈ queen*. Once you see that, RAG stops feeling like magic and starts feeling like geometry. ✨

This is the same pattern behind Notion AI, Perplexity, Glean, and "Chat with your codebase" features in modern IDEs. Big confidence boost knowing I can clone the architecture in ~50 lines.

Learning AI Engineering with @Shivank Agrawal at @Scaler — Day 3 done. The pace of going from *I've heard of LLMs* to *I can ship AI products* is genuinely wild.

👉 What PDF should I point it at next? Drop a suggestion in the comments.

#AIEngineering #RAG #LLM #Python #BuildInPublic #100DaysOfCode

👍 You and others 💬 Comments ↗ Repost

👉 When you type "@Shivank Agrawal", pick linkedin.com/in/shivankagrawal so the tag links correctly. Tagging your instructor and Scaler boosts reach & lets us amplify your post.

🎙️ Speaker note · the post-class ritual

Same as before: **10 minutes in class** for everyone to post. Then "follow-and-like" round across the cohort. Three classes in, this is becoming muscle memory — and that's exactly

Want a variation? Pick a flavour, swap the PDF

The recipe is the same; the data makes it interesting. Try one of these on your own:

Chat with the Constitution

Index the Indian Constitution PDF. Ask "what are the Fundamental Rights?" with citations.

data: indiacode.nic.in

Chat with an earnings call

Index TCS or Infosys' last quarterly transcript. Ask about margins, guidance, hiring.

data: investor relations sites

Chat with your textbook

Index a chapter. Ask exam-style questions. Suddenly: a personalised tutor.

data: your bookshelf

Chat with company HR policy

Genuinely useful at your workplace. "How many leaves do I have left?" "What's the WFH policy?"

data: your HR portal

BLOCK 23 · ~5 MIN · WHAT'S COMING

23

Where this is *going*

RAG works. But the frontier is making it *smarter* — and that's where this course leads next.

Agentic RAG — when retrieval becomes a decision

Class 2's agent loop with Class 3's library. The agent *decides*: "do I need to look this up? Is what I found enough? Should I rewrite my query and try again?"

The loop you'll meet next class

Query → retrieve → grade chunks → if bad, rewrite query, search again → if still bad, ask user for clarification → answer. Same agent loop you built in Class 2, with the librarian (Class 3) as one of its tools.

Advanced RAG — the tricks the senior engineers use

HyDE

Hypothetical Document Embeddings: have the LLM *imagine* what the answer would look like, then search for chunks that match the imagined answer. Counter-intuitive, works surprisingly well.

Step-back prompting

Before searching, ask the LLM to generalize the question. "What's the formula for compound interest in this case?" → "What is compound interest?" → broader, better retrieval.

Graph RAG

Build a knowledge graph from your docs (entities + relationships). Now you can answer "who reports to X" by walking the graph, not just searching.

RAG evaluation

How do you measure if your RAG is good? Three metrics: *relevance*, *faithfulness*, *correctness*. We'll wire up a real eval pipeline.

Where you are right now

Three classes in, you've built a chatbot, an agent, and a RAG system. You understand the four pieces of every AI product — *model · prompt · tool · retrieval*. Almost everything ahead is recombining these four in cleverer ways. You're past the steep part of the curve.

Embeddings

Text → vectors. Similar meaning = nearby.

Chunking

The unglamorous knob that decides quality.

Vector DB

Three calls: add, query, delete.

🎯

You shipped

A real "Chat with your PDF" — on LinkedIn. 🎉

You shipped class 3 🎉

You now understand the four pieces of every modern AI product. Most people who claim to "know AI" don't know what you know.

FOR NEXT CLASS

Index a PDF that matters to *you* (your resume, your notes, your company policy) and bring 3 questions where your RAG surprised you — for better or worse. We'll use those to motivate Agentic RAG.
